# Proyecto III, Módulo V: AI Engineering

## AgentSquad: Sistema multi-agente autónomo para investigación de mercado y automatización de procesos con herramientas de código

## Situación profesional

Forman parte del equipo de *AI Engineering* de una consultora que diseña soluciones de automatización inteligente para pymes, startups y organizaciones del tercer sector. Sus clientes necesitan tomar decisiones estratégicas, como abrir un nuevo local, lanzar un producto o entrar en un mercado. Para eso dependen de estudios de mercado que hoy se hacen a mano: buscar información en la web, revisar competidores, recopilar precios, analizar datos y redactar informes. El proceso tarda semanas, cuesta mucho y queda desactualizado enseguida.

Su misión es construir un **sistema multi-agente autónomo**. En él, varios agentes de IA especializados colaboran entre sí, razonan, deciden qué herramientas usar y ejecutan código para completar de punta a punta una investigación de mercado. Además, el sistema debe automatizar al menos un proceso posterior, como generar el informe, actualizar una hoja de seguimiento o preparar un correo para el cliente.

Su equipo actuará como una *squad* de producto y podrá elegir el sector o caso de uso. Algunos ejemplos de inspiración:

- **Apertura de negocio local:** analizar la viabilidad de abrir una cafetería de especialidad, un coworking o una academia en un barrio concreto (competencia, precios, reseñas, perfil del público).
- **Lanzamiento de producto digital:** estudiar el mercado de una app o SaaS (competidores, modelos de precios, funcionalidades diferenciales, tendencias).
- **Monitor de competencia en e-commerce:** vigilar periódicamente precios, catálogo y posicionamiento de competidores y generar alertas y recomendaciones.
- **Entidad social:** identificar financiadores, aliados potenciales y tendencias del sector para una ONG que quiere lanzar un nuevo programa.

### Problema / Necesidad

Los LLM conversacionales responden preguntas, pero no *trabajan*: no planifican, no consultan fuentes externas por sí solos, no ejecutan cálculos fiables ni coordinan tareas en varias etapas. La organización cliente necesita:

- **Descomponer una tarea compleja** (una investigación de mercado) en subtareas asignadas a agentes con roles especializados que colaboran y se revisan entre sí.
- **Actuar en el mundo real mediante herramientas:** buscar en la web, leer documentos, ejecutar código Python para analizar datos y generar gráficos, en lugar de "inventar" cifras.
- **Controlar el flujo:** las decisiones del sistema deben seguir un grafo de estados explícito y auditable, con bucles de revisión y condiciones de parada que eviten bucles infinitos y costos descontrolados.
- **Mantener a la persona en el centro (human-in-the-loop):** las acciones con impacto, como enviar, publicar o sobrescribir, requieren aprobación humana explícita.
- **Garantizar trazabilidad y veracidad:** cada afirmación del informe final debe poder rastrearse hasta su fuente o hasta el código que la produjo.
- **Estandarizar la conexión con herramientas** mediante el Protocolo de Contexto de Modelos (MCP), para que el sistema sea extensible.

### Producto final esperado

Una solución de software funcional compuesta por:

1. **Equipo de agentes (CrewAI):** mínimo **tres agentes** con rol, objetivo y herramientas definidos. Por ejemplo: *Investigador/a* (búsqueda y extracción), *Analista de datos* (ejecuta código Python para procesar datos y generar gráficos), *Redactor/a estratégico/a* (elabora el informe) y, opcionalmente, *Revisor/a crítico/a* (verifica fuentes y coherencia).
2. **Orquestación basada en estados (LangGraph):** un grafo que modele el flujo de trabajo con al menos un **nodo condicional** (enrutamiento según el resultado), un **bucle de revisión** con límite de iteraciones y un **punto de interrupción human-in-the-loop** antes de cualquier acción de automatización.
3. **Herramientas externas (Tool Use):** al menos **tres herramientas**, de las cuales:
    - una debe ser de **ejecución de código** (análisis con pandas, gráficos con matplotlib/plotly),
    - una debe ser **desarrollada por el equipo** (función propia con esquema de entrada validado con Pydantic),
    - y al menos una debe conectarse mediante un **servidor MCP** (existente o propio).
4. **Componente multimodal:** al menos un agente debe **analizar imágenes**, como capturas de webs de competidores, fotos de producto o los gráficos generados por el propio sistema, e incorporar esa interpretación al informe.
5. **Automatización del proceso:** el sistema debe ejecutar al menos una acción posterior a la investigación, como generar un informe en Markdown/PDF, actualizar un CSV o una base de datos de seguimiento, o preparar un borrador de correo, siempre con aprobación humana previa.

6. **Interfaz de usuario:** un frontend (Streamlit o Gradio) donde la persona usuaria define el encargo, ve en tiempo real qué agente trabaja y qué herramienta usa, aprueba o rechaza acciones y descarga el informe final con sus fuentes.
7. **(Reto avanzado opcional) Agente de voz:** permitir dar el encargo por voz o escuchar un resumen hablado del informe.

## Conocimientos técnicos

- Fundamentos de agentes de IA: bucles de decisión autónomos, razonamiento y el patrón ReAct.
- Uso de herramientas externas (*Tool Use* / *function calling*) e integración de código ejecutable.
- Arquitecturas multi-agente: roles, delegación, procesos secuenciales y jerárquicos con **CrewAI**.
- Flujos de trabajo basados en estados y grafos con **LangGraph**: nodos, aristas condicionales, estado compartido, persistencia e interrupciones.
- **Model Context Protocol (MCP):** arquitectura cliente-servidor y conexión estandarizada de herramientas.
- Modelos multimodales: análisis de imágenes y gráficos.
- Salidas estructuradas y validación de tipos con Pydantic.
- Gestión de costos, tokens, *rate limits* y control de bucles.

## Competencias evaluadas

- **C9. Desarrollo y optimización de arquitecturas de Deep Learning.**
  **Nivel 3 (Transponer):** Desarrolla sistemas multi-agente que demuestren razonamiento autónomo y uso de herramientas externas. *(Taxonomía de Bloom: Evaluar y Crear).*
- **C2. Desenvolverse en entornos técnicos y digitales básicos para el desarrollo de IA.**
  **Nivel 3 (Transponer):** Aprende a utilizar una herramienta nueva sin guía directa (MCP), integra varias herramientas en su flujo de trabajo (CrewAI, LangGraph, MCP, modelos multimodales) y justifica su elección. *(Taxonomía de Bloom: Aplicar y Evaluar).*
- **C8. Desarrollar un proyecto aplicado en equipo y comunicar resultados técnicos.**
  **Nivel 3 (Transponer):** Impulsa el proyecto con autonomía dentro del equipo y lo presenta como una propuesta de valor profesional, defendiendo decisiones técnicas y éticas. *(Taxonomía de Bloom: Aplicar y Evaluar).*
- **C10. Aplicar criterios éticos, sociales y de gobernanza en el desarrollo y uso de la IA.**
  **Nivel 3 (Transponer):** Decide cuándo no es adecuado delegar una decisión en un agente autónomo y lo argumenta. Justifica la arquitectura ante terceros,

asegurando que las decisiones de los agentes sean explicables, trazables y supervisadas. *(Taxonomía de Bloom: Evaluar)*.

### Criterios de rendimiento y evaluación:

- **Diseño de roles y responsabilidades:** cada agente tiene un rol, un objetivo y un conjunto de herramientas acotado y justificado, sin solapamientos innecesarios.
- **Autonomía controlada:** el grafo de estados define explícitamente cuándo el sistema avanza, repite o se detiene. Existen límites de iteraciones y de costo, y el sistema no entra en bucles infinitos.
- **Uso efectivo de herramientas:** los datos cuantitativos del informe provienen de la ejecución de código o de fuentes consultadas, no de la "memoria" del modelo.
- **Supervisión humana:** las acciones de automatización con impacto se interrumpen hasta recibir aprobación explícita en la interfaz.
- **Trazabilidad y explicabilidad:** el sistema registra qué agente hizo qué, con qué herramienta y sobre qué fuente. El informe final cita sus fuentes.
- **Integración de MCP y multimodalidad:** al menos una herramienta se conecta vía MCP y el análisis de imágenes aporta valor real al resultado, no es decorativo.
- **Arquitectura colaborativa:** el repositorio refleja trabajo equilibrado mediante ramas y pull requests, evidenciando metodologías ágiles.

### Modalidades de evaluación (Checklist)

#### I. Diseño del sistema multi-agente

- Definición de al menos tres agentes con rol, objetivo, *backstory* y herramientas asignadas (CrewAI).
- Diagrama de arquitectura del sistema (agentes, herramientas, flujo) incluido en el README.
- Justificación del tipo de proceso elegido (secuencial, jerárquico o híbrido).

#### II. Orquestación y control de flujo

- Grafo de estados funcional en LangGraph con estado compartido tipado.
- Al menos una arista condicional (enrutamiento) y un bucle de revisión con límite de iteraciones.
- Punto de interrupción human-in-the-loop operativo antes de la automatización.
- Manejo de excepciones: qué hace el sistema si una herramienta falla, si no encuentra información o si se agota el presupuesto de tokens.

#### III. Herramientas, MCP y multimodalidad

- Herramienta de ejecución de código funcional (análisis de datos y generación de gráficos).
- Herramienta propia con esquema de entrada validado con Pydantic.
- Conexión de al menos una herramienta a través de un servidor MCP.

- Agente con capacidad multimodal que interpreta imágenes o gráficos e integra el resultado en el informe.

**IV. Automatización e interfaz**

- Ejecución de al menos una acción automatizada posterior (informe, CSV/BD, borrador de correo).
- Interfaz web que muestra el progreso de los agentes, permite aprobar o rechazar acciones y descargar el informe.
- Informe final con fuentes citadas y registro (log) de las acciones de cada agente.

**V. Trabajo en equipo y buenas prácticas**

- Repositorio organizado con requirements.txt (o pyproject.toml), .env.example sin claves expuestas y README técnico completo.
- Uso adecuado de Git (commits descriptivos, ramas, pull requests revisados).
- (Opcional) Agente de voz funcional.

## Presentación Oral

Duración orientativa: 7-10 minutos de presentación. Debe incluir:

- **Caso de uso y valor de negocio:** qué problema resuelven, para quién y cuánto tiempo o costo ahorra frente al proceso manual.
- **Demo:** ejecución de un encargo real de punta a punta, mostrando la colaboración entre agentes, el uso de herramientas y el punto de aprobación humana.
- **Defensa técnica:** por qué eligieron esos roles, esa topología de grafo, esas herramientas y ese modelo (comercial o local). Qué aportan CrewAI y LangGraph en su solución y qué limitaciones encontraron.
- **Aprendizaje autónomo:** cómo aprendieron e integraron MCP sin guía directa.
- **Reflexión ética y de gobernanza:**
    - ¿Qué decisiones *no* delegaron en los agentes y por qué?
    - Riesgos de la autonomía: alucinaciones en cifras, *prompt injection* a través de contenido web, acciones no deseadas, costos descontrolados.
    - Respeto de los términos de uso y privacidad de las fuentes consultadas.
    - Cómo puede un tercero auditar las decisiones del sistema.

## Modalidad Pedagógica

- **Modalidad:** trabajo grupal (squads de 3–4 personas).
- **Duración estimada:** 2 semanas
- **Metodología:** Agile con sprints semanales, dailies breves, revisión entre pares y *pair/mob programming*.
- **Acompañamiento:** checkpoints guiados por el equipo formativo
- **Evaluación:** checklist técnica + presentación oral + revisión del repositorio. Coevaluación.

## Recursos y apoyos

- Documentación oficial de CrewAI: https://docs.crewai.com/
- Documentación oficial de LangGraph: https://docs.langchain.com/oss/python/langgraph/overview
- Documentación oficial de Model Context Protocol: https://modelcontextprotocol.io/
- Anthropic, *Building effective agents* (patrones de diseño de agentes y cuándo no usarlos): https://www.anthropic.com/engineering/building-effective-agents
- Artículo recomendado: Yao et al. (2022), *ReAct: Synergizing Reasoning and Acting in Language Models*: https://arxiv.org/abs/2210.03629
- OWASP Top 10 para aplicaciones LLM (riesgos de seguridad, incluido *prompt injection* y *excessive agency*): https://genai.owasp.org/llm-top-10/
- Materiales y notebooks de las píldoras de la Fase 3 del Módulo V.

## Glosario de Términos

- **Agente de IA:** sistema basado en un LLM que percibe su entorno, razona sobre un objetivo, decide qué acciones ejecutar (incluido el uso de herramientas) y evalúa el resultado en un bucle hasta completar la tarea.
- **Sistema multi-agente:** arquitectura en la que varios agentes especializados colaboran, se delegan tareas o se revisan entre sí para resolver un problema complejo.
- **ReAct (Reasoning + Acting):** patrón en el que el modelo alterna pasos de razonamiento y acciones (llamadas a herramientas), usando las observaciones obtenidas para decidir el siguiente paso.
- **Tool Use / Function Calling:** capacidad de un LLM para invocar funciones externas con argumentos estructurados, como buscar en la web, ejecutar código o consultar una base de datos.
- **CrewAI:** framework para definir equipos ("crews") de agentes con roles, tareas y procesos de colaboración.
- **LangGraph:** framework de orquestación que modela flujos de agentes como grafos de estados, con nodos, aristas condicionales, persistencia e interrupciones.
- **Estado (State):** estructura de datos compartida que recorre el grafo y que cada nodo puede leer y actualizar.
- **Human-in-the-loop:** mecanismo que pausa la ejecución para que una persona revise, apruebe o corrija una decisión antes de continuar.
- **MCP (Model Context Protocol):** estándar abierto que define cómo las aplicaciones de IA se conectan con herramientas y fuentes de datos externas mediante una arquitectura cliente-servidor.
- **Modelo multimodal:** modelo capaz de procesar e integrar distintos tipos de entrada, como texto, imágenes o audio.
- **Excessive agency (agencia excesiva):** riesgo de que un agente tenga más permisos o autonomía de los necesarios y ejecute acciones dañinas o no deseadas.

- **Prompt injection:** ataque en el que contenido malicioso (por ejemplo, en una página web consultada) introduce instrucciones que manipulan el comportamiento del agente.